

Sistemas Distribuídos

Processos e sincronização

REALIZAR APÓS posição 7: Sistemas Distribuídos - Aula 6

Posições na playlist 1 a 7 | 120 minutos | 100 pontos

Curso principal: COS470 - Sistemas Distribuídos |

https://www.youtube.com/watch?v=TEEy5f46h_Q&list=PLP0bYj2MTFcuXa4-EbBKhvhr-rkxpeR8&index=1

Conteúdo verificável: IPC; threads; locks; Peterson; atomicidade; semáforos; monitores

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Comunicação e memória

Compare IPC por memória compartilhada e troca de mensagens em processos locais. Explique como threads mudam o compartilhamento e por que o mesmo algoritmo não pode assumir uma variável compartilhada comum entre computadores independentes.

Desenvolvimento / justificativa / esquema:

Peterson

Para dois participantes i e j , analise: `flag[i]=true; turn=j; while(flag[j] and turn==j):esperar; seção crítica; flag[i]=false`. Mostre execução com intenções simultâneas, explique exclusão mútua sob leituras/escritas atômicas e ordem sequencial, e indique por que essas hipóteses importam.

Desenvolvimento / justificativa / esquema:

Test-and-set

Defina `test_and_set` como troca atômica que retorna valor antigo e escreve `true`. Construa spinlock e explique por que separar leitura e escrita comuns não funciona. Discuta justiça e tempo gasto esperando.

Desenvolvimento / justificativa / esquema:

Semáforo e monitor

Projete fila limitada para produtores/consumidores usando monitor com condições `nao_cheia` e `nao_vazia`. Dê pseudocódigo com `while`, explique a necessidade de testar novamente após despertar e compare com semáforos contadores.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Comunicação e memória (15 pontos)

Memória compartilhada permite leituras/escritas diretas com sincronização; mensagens exigem interface/protocolo e cópia/serialização conforme mecanismo. Threads compartilham endereço no processo; máquinas independentes têm memórias distintas e comunicação sujeita a atraso/falha. Uma flag local não é automaticamente observada remotamente. Crédito exige explicitar estado, visibilidade e coordenação, sem reduzir distribuição apenas a quantidade de threads.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Peterson (25 pontos)

Ambos ativam flags; a última escrita em turn favorece o outro, portanto no máximo um passa enquanto ambos interessados. Se ambos estivessem dentro, condições exigiriam simultaneamente $\text{turn} \neq j$ para i e $\text{turn} \neq i$ para j , impossível com dois valores. Ao sair, flag false permite avanço. Reordenação/visibilidade em hardware/linguagem pode invalidar prova sem garantias adequadas. Espera ocupada consome recursos; a análise não é autorização para implementar qualquer variável comum como trava em plataforma moderna.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Test-and-set (25 pontos)

```
while test_and_set(lock): esperar  
seção crítica  
lock = false    # liberação com garantia adequada de visibilidade
```

Apenas um recebe valor antigo false. Com leitura/escrita separadas, ambos podem ler false antes de gravar true. Spinlock não garante justiça e usa CPU durante espera; adequação depende de seção curta e contexto. Atômico no teste não dispensa semântica correta da liberação e dos dados protegidos.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Semáforo e monitor (35 pontos)

```
inserir(x): # dentro do monitor
    while fila cheia: wait(iao_cheia)
    fila.adicionar(x)
    signal(iao_vazia)
retirar():
    while fila vazia: wait(iao_vazia)
    x = fila.remover()
    signal(iao_cheia)
    return x
```

wait libera exclusão do monitor e a readquire antes de retornar; condição pode não continuar verdadeira ao despertar, exigindo while sob semântica comum de sinalização. Semáforo mantém contador de permissões; condição de monitor não é estoque de sinais. Invariante $0 \leq \text{tamanho} \leq N$ e alteração protegida são obrigatórios. Aceitar semântica diferente se claramente declarada e prova ajustada.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.